Held-out Accuracy Is Not New-Collection Accuracy: A Cross-Collection Benchmark of Poultry Dropping Classifiers and a Process-Isolated Pen-Camera Pipeline on a QNX Raspberry Pi 5

Deekshith Reddy Bhoomireddy Rishit Rai Lance Streuber

Abstract

Deep networks classify photographs of poultry droppings into healthy, coccidiosis, salmonella and Newcastle disease with reported accuracies of 97.8 to 99.3%, but these numbers come from random splits of a single collection. We retrained the classifiers earlier work used (MobileNetV2 and EfficientNet-B0) and our YOLO11n detector on the same split of the public Tanzanian farm photographs. We then tested all of them on the same 679 held-out farm photographs and on 1,255 photographs from a separate collection, taken on farms and labelled from PCR tests. Every model scores 98.5 to 99.6% on the farm test and 75 to 78% on the PCR-labelled photographs when it always gives an answer, a drop of 21 to 25 points, and the detector does not differ significantly from either classifier. The farm collection may explain part of this, because its salmonella and Newcastle photographs come from experimentally inoculated birds. An audit found 49 farm test photographs (7.2%) that are exact copies of training photographs, and 19 pairs of identical PCR files filed under two classes, but neither changes accuracy by more than about one point. The detector's box score ranks photographs well but is not a calibrated probability. We also graded every PCR-labelled photograph for appearance with three AI raters (Krippendorff's alpha 0.76). Salmonella droppings look almost healthy, yet the models still name them correctly, while normal looking coccidiosis is mostly missed. Because a fixed pen camera sees a dropping as a small object, we fine-tuned the detector on public barn video with real droppings pasted onto the litter, split by camera. On unseen cameras, recall rises from 0.29 to 0.80 and precision from 0.11 to 0.93, at a cost of 0.02 mAP50 on close-ups. A pipeline built around the close-up detector runs on a Raspberry Pi 5 under the QNX microkernel. Each stage is a separate process, and a stage killed with `SIGKILL` was relaunched within 20 to 33 ms of the supervisor noticing the exit while the others kept running. We release the code, the per-photograph predictions and every figure with its data.

1 Introduction

Coccidiosis alone was estimated to cost the global chicken industry about £10.4 billion at 2016 prices [5]. Coccidiosis, salmonellosis and Newcastle disease change the colour and consistency of droppings, and farmers already look at droppings to judge flock health. This has made dropping photographs a popular target for deep learning. On the public Tanzanian collection of Machuve et al. [24, 25] and what appear to be copies of it, convolutional classifiers report 97.8 to 99.3% accuracy [10, 11, 17, 25, 29], and several groups have proposed phone, web or edge systems around them [10, 11, 30].

These accuracies come from random splits of a single collection. A random split by photograph can place near-identical pictures on both sides, as documented for standard benchmarks [2], and leakage of this kind has inflated results in many fields [19]. The collection has a second property that matters more. Its healthy and coccidiosis photographs were taken on farms, while its salmonella and Newcastle photographs were taken from experimentally inoculated birds [24], so the setting of a photograph can stand in for its class. The original authors wrote that they did not expect their model to be affected by distribution shift [25]. Only one study has tested the four-class task on a second collection, a local-farm set whose labelling is not described in its abstract [30], and none on labels from laboratory tests. The same group released a second collection, taken on farms and labelled from PCR results [23], which offers that test. A second gap is the camera. Published systems classify close-up photographs taken by a person. A camera fixed above a pen sees each dropping as a small object among birds, feeders and litter, and nobody is there to restart the device when software fails.

We address both gaps with one detector and one honest protocol. Our contributions are as follows.

1. A benchmark of the classifiers earlier work used (MobileNetV2, EfficientNet-B0), retrained on the same split as our YOLO11n detector and tested on the same 679 farm photographs and on 1,255 PCR-labelled photographs that no model saw. Every model scores 98.5 to 99.6% on the farm test, in line with published numbers, and 74.6 to 78.2% on the PCR-labelled photographs, a drop of 21 to 25 points for every model. Our detector does not differ significantly from either classifier on the second collection. To our knowledge this is the first test of the four-class task on PCR-labelled photographs with the split, the models and

every per-photograph prediction released.

2. An audit of both collections. In the farm collection, 49 of the 679 test photographs (7.2%) are exact copies of training photographs, but removing them changes the farm accuracy by at most 0.1 points. In the PCR collection, 19 pairs of byte-identical files are filed under two different classes. We also show that the detector's box score is not a calibrated probability.
3. A pen-scale fine-tune trained with real droppings pasted onto barn video frames, split by camera. On cameras it never saw, recall rises from 0.29 to 0.80 and precision from 0.11 to 0.93, at a cost of 0.02 mAP50 on close-ups.
4. A pipeline running on a Raspberry Pi 5 under the QNX microkernel, with our detector and pen model timed on the device. Each stage is a separate process. When we killed a stage with `SIGKILL`, the supervisor relaunched it within 20 to 33 ms of noticing the exit, while the others kept running. To our knowledge no peer-reviewed work has run poultry health vision on a microkernel real-time operating system.

We release the code, the per-photograph predictions, the duplicate hashes and every figure with its data.

2 Related work

Classifying poultry droppings. Machuve et al. collected the Tanzanian photographs we use and compared five convolutional networks [25]. With fine-tuning, MobileNetV2 reached 98.02% and Xception 98.24% on a farm-labelled test set, and the authors recommended MobileNetV2 for deployment. Later work on what appear to be the same photographs, often through a 224-pixel Kaggle copy, reports similar numbers. Degu and Simegn combined a YOLOv3 detector with a ResNet-50 classifier for a phone application and report 98.7% [10]. Reyad et al. report 99.25% with a majority-vote ensemble [29], and Huda et al. 97.78% with a hybrid of EfficientNetV2-B0 and DenseNet-121 [17]. The closest pipeline to ours is that of Dhungana et al., a YOLO11n detector followed by an EfficientNet-B0 classifier, with a detector mAP50 of 0.881 and a classifier accuracy of 99.12% [11]. Sakib et al. capture photographs with an ESP32-CAM and classify them with EfficientNet-B0 on a server, reporting 99.19% on a public test set and 95.62% on a separate local-farm set [30]. Other groups built their own data. Aworinde et al. released 14,618 cleaned Nigerian smartphone photographs at 100×100 pixels [1], and Nakrosis et al. classified 487 Lithuanian photographs of droppings on litter into six appearance classes, with 91.78% accuracy [26]. Wang et al. detected droppings on a conveyor belt below caged broilers [33]. Li surveys poultry datasets [22].

Testing beyond one collection. Bodempudi et al. are the closest to our evaluation. They classified droppings as *Salmonella* PCR positive or negative across United States and African images and report up to 85% across regions [6]. Their task is binary and their test set is a different region. We test all four classes, on a collection from the same group whose labels were confirmed in the laboratory, which separates the effect of label source and collection from the effect of region. More broadly, leakage between training and test data has inflated results across many fields [19], near-duplicate images between the splits of standard benchmarks have been documented [2], and networks often learn shortcuts such as framing or background that stand in for the class [13]. We use the calibration and selective prediction tools of Guo et al. [15] and Geifman and El-Yaniv [12].

Edge deployment and operating systems. Livestock vision on small devices usually means shrinking the model, for example by up to 270 times for microcontrollers [35]. Published dropping systems run on a phone, a server or a laptop [10, 11, 30]. None that we found addresses what happens when part of the software fails on an unattended device. Microkernels move drivers and services into isolated processes, so a fault stays in one process [3, 20]. A vendor white paper compares QNX 8 with real-time Linux on a Raspberry Pi 5 with a neural accelerator and reports similar throughput with less variation [4]. It is not peer reviewed and does not measure fault recovery. We measure recovery directly.

3 Methods

3.1 Data

Farm photographs. We used the poultry faecal image collection of Machuve et al. released on Zenodo (record 4628934, CC BY 4.0) [24, 25]. It contains 6,812 full-resolution smartphone photographs taken in Arusha and Kilimanjaro, Tanzania, with official bounding boxes in YOLO format for four classes: healthy, coccidiosis, salmonella and Newcastle disease. According to the record, the healthy and coccidiosis photographs were taken on poultry farms, the salmonella photographs from experimentally inoculated birds one week after inoculation, and the Newcastle photographs from inoculated birds within three days. Each class therefore comes with its own setting. We downloaded the archives through the Zenodo API and checked each against the MD5 checksum the publisher lists. Following the dataset, we refer to the disease classes as *coccidiosis-like*, *salmonella-like* and *Newcastle-like*, because no photograph in this collection carries a laboratory diagnosis of its own dropping.

Lab-confirmed photographs. The same group released a second, separate collection of 1,255 photographs from layer, cross-bred and indigenous chickens on farms in the same two regions, each coded to a faecal sample that the dataset authors tested by polymerase chain reaction

(PCR) (Zenodo record 5801834) [23]. We call them PCR-labelled or lab-confirmed photographs. They contain 373 coccidiosis, 347 healthy, 349 salmonella and 186 Newcastle disease photographs. No model in this study was trained, tuned or selected on any of them. We use them only as an external test set. The release contains 35 pairs of byte-identical files, so it holds 1,220 distinct images. In 19 of these pairs the two files sit in different classes (16 coccidiosis and healthy, 3 healthy and salmonella), so at least 19 labels or file placements are wrong. We report results on all 1,255 files, as released, and give the effect of removing the 38 conflicting files in Section 4.1.

Barn video. To teach the detector the size droppings have in a fixed pen camera, we took background frames from two public video collections of broiler houses: the PIO videos (Zenodo record 16686320, CC BY 4.0) [7, 8] and the NESTLER poultry videos (Zenodo record 15063494, CC BY 4.0) [27]. No dropping in these videos is labelled, and we did not label any.

3.2 Split

The 6,809 farm photographs that have labels (three label files are empty) were split 80/10/10 by photograph within each class, using a fixed seed on the sorted file names, giving 5,451 training, 679 validation and 679 test photographs. This is the kind of random split earlier work on this collection reports. The detector and every baseline in Section 3.5 use exactly this split. To measure leakage, we reduced every photograph to a 256-bit difference hash [21] and recorded, for each test photograph, the Hamming distance to the closest training photograph.

3.3 Detector

Our detector is YOLO11n [18] with four classes, fine-tuned from COCO weights at 640 pixels with early stopping (patience 20, seed 0) on a local laptop GPU. The Newcastle-like class has about a sixth of the photographs of the others, so its training photographs are repeated three times, after the split and in the training set only. Training stopped at epoch 69 and the weights of epoch 49, the best on validation, were kept (`dropping4-laptop`). We exported it to ONNX at 320, 416 and 640 pixels. To check that the result does not depend on the machine, we repeated the training on the Veri cloud platform on one NVIDIA A10G GPU, with batch 32 and a 30-minute limit (20 epochs). That run reached a test mAP50 of 0.854 and mAP50-95 of 0.573, against 0.873 and 0.573 for the local run, so the two are close. All results below use the local run.

To compare the detector with photograph classifiers, we score a whole photograph with it: the photograph is letterboxed to 640 pixels, every candidate box keeps its class scores, and each class takes its best score in the photograph. The photograph's call is the class with the highest score, or *no detection* when no box reaches 0.25, the confidence threshold the deployed pipeline uses.

3.4 Pen-scale fine-tuning

We handle pen frames at 1280×720 and assume that a dropping spans 8 to 30 pixels along its longest side there. This is a design assumption for a ceiling camera over a pen, not a measurement, since no droppings in our pen videos are labelled. In the farm collection, by contrast, the disease-class boxes cover a median of 12 to 22% of the photograph. We built a pen-scale training set by copy-paste augmentation [14]: real labelled droppings were cut from the farm photographs along their official boxes, shrunk to pen size, and pasted onto empty litter in barn video frames. A dropping from a training photograph is only ever pasted into a training tile, and likewise for validation and test, so the farm split carries over. Background frames were split *by camera*, never by frame. Five PIO videos and the remaining NESTLER cameras form the training set, one NESTLER camera the validation set, and three NESTLER cameras plus the PIO video used in our live demonstration the test set. Frames with birds and equipment and no pasted dropping were kept as negatives. The model `dropping4-pen` was fine-tuned from `dropping4-laptop` on these tiles mixed with the original farm photographs, for up to 40 epochs (patience 10). It stopped at epoch 38.

We state the limitation plainly: the pen test is synthetic. It measures whether the model finds a real dropping of pen size on litter and cameras it has never seen, not field accuracy on naturally occurring droppings.

3.5 Baselines

Published results on this collection come from different splits, image sizes and protocols, so we retrained the classifiers earlier work used instead of quoting their numbers. MobileNetV2 [31] is the model Machuve et al. recommended for deployment [25], EfficientNet-B0 [32] is the classifier stage of a recent YOLO11 and EfficientNet-B0 pipeline [11]. Each was initialised from ImageNet weights and trained on the same 5,451 training photographs, resized to 224×224 without keeping the aspect ratio (as in the 224-pixel copy of the collection most earlier work used), with AdamW and a one-cycle learning rate schedule. MobileNetV2 was trained for 20 epochs and EfficientNet-B0 for 8, both with one seed, because of the compute time available to us. Augmentation was random crops, horizontal and vertical flips, and small brightness and contrast changes. We did not change hue, because colour is the main visual sign of disease. The epoch with the best validation accuracy was kept. Both passed 98% validation accuracy within four epochs, so the shorter EfficientNet-B0 run is unlikely to change the comparison, but one seed cannot measure run-to-run variation.

3.6 Evaluation

All models are scored on the same 679 farm test photographs and the same 1,255 lab-confirmed photographs. We report accuracy, per-class recall and macro-F1. Proportions carry 95% Wilson score intervals [34], which behave well near 0 and 1. For the detector we also report a risk-coverage curve [12], the accuracy of the answers when the model only answers above a confidence threshold, and a reliability diagram with the expected calibration error over ten equal-width bins [15].

3.7 Appearance grades for the lab-confirmed photographs

PCR says whether a pathogen is in the sample, not how the dropping looks. To ask whether the models fail on droppings that look normal, we graded every one of the 1,255 lab-confirmed photographs for appearance. A written rubric scores six visual variables 0, 1 or 2, or leaves them empty when the photograph does not show them: form, the white urate cap, colour, blood, mucus or foam, and undigested feed. These are the signs used in clinical checks of droppings. The grade is computed, never chosen, as $\max(1, 10 - p)$ where p is the sum of the six scores with blood counted twice, so every lost point has a named reason. Grades 9 and 10 mean the dropping looks normal.

The raters were three AI vision models, not people: Claude Opus, Claude Sonnet and Claude Haiku, each run with the same written instructions and no shared context. Each photograph got a random identifier and was re-encoded at 768 pixels with no metadata, and the key linking identifiers to file names and PCR classes stayed outside the raters' folder until grading ended. Each rater saw the photographs in its own random order. 126 photographs (10%) were shown a second time under a new identifier to measure each rater's agreement with itself. The released sub-score for a photograph is the median of the raters who could judge it, needing at least two, and its grade is recomputed from those medians. We report Krippendorff's alpha for ordinal data across the three raters [16] and quadratic weighted kappa for each pair and for each rater against itself [9].

3.8 Deployment on QNX

The deployed system runs on a Raspberry Pi 5 with a Camera Module 3 under QNX 8.0, a microkernel real-time operating system [3]. The pipeline is split into separate processes (sections), each with its own memory and a fixed real-time priority: camera capture in C, a still-spot finder that flags new objects that appeared and stayed still, the detector, an explanation step, and links to the dashboard and a wrist-worn display. Full frames stay in a shared-memory ring of four slots, and sections exchange only small messages through QNX native message passing. A supervisor restarts any section that exits. On the Pi the detector is the close-up model, run on a full-resolution patch around each candidate from the spot finder, not the pen model. At pen scale the close-up model can score white birds as salmonella, so the explanation step dismisses a candidate unless its box lies on the spot, has the size of a dropping, is not white, is not lighter than the floor around it, and scores at least 0.6. Inference uses ONNX Runtime [28] 1.24.4 from the QNX open-source package repository, called from a small C bridge because the repository's Python binding does not load on QNX (Section 5). We report the detector's inference time on the Pi with four threads, without image decoding or post-processing.

Alerts reach people in two ways. A wrist-worn display polls the dashboard and shows each pen's state. A separate messaging service sends pen alerts to the farmer over iMessage through the Photon platform and answers follow-up questions in plain language with a large language model (DeepSeek). Each alert states what the camera recorded, the model's closest match with its score, and that it is not a laboratory test. The service never tells the farmer to treat or cull birds. It contacts a veterinarian only after the farmer clearly agrees, and that decision is made by tested rules in the application code, not by the language model. We describe this service for completeness and do not evaluate it here. Figures 1 and 2 show the hardware and the three interfaces.

4 Results

4.1 All models drop on lab-confirmed photographs

Table 1 and Figure 3 compare every model on the two test sets. On the farm test all of them are in the range earlier work reports. MobileNetV2 calls 99.6% of the 679 photographs correctly (95% CI 98.7 to 99.8%), EfficientNet-B0 99.0% (97.9 to 99.5%), and our detector 98.5% (97.3 to 99.2%) when it always gives its best guess. On the 1,255 lab-confirmed photographs every model loses more than 20 points. MobileNetV2 falls to 74.6% (72.1 to 76.9%), EfficientNet-B0 to 78.2% (75.9 to 80.4%) and the detector to 76.3% (73.9 to 78.6%). For each model the intervals of the two test sets are far apart, so the drop is not sampling noise.

The drop is 25.0 points for MobileNetV2, 20.8 for EfficientNet-B0 and 22.2 for the detector. Our detector does not classify better than the classifiers earlier work used. A paired exact McNemar test on the 1,255 lab-confirmed photographs finds no difference between the detector and MobileNetV2 ($p = 0.20$) or EfficientNet-B0 ($p = 0.16$), while EfficientNet-B0 is ahead of MobileNetV2 ($p = 0.002$). Removing the 38 files that the release places in two classes raises every model by about one point (MobileNetV2 75.6%, EfficientNet-B0 79.5%, detector 77.3%), so the conflicting labels do not explain the



Figure 1: The deployed hardware. (a) Raspberry Pi 5 running QNX 8.0 with a Camera Module 3 on its ribbon cable and an active cooler. (b to d) The wrist-worn display (M5Stack StopWatch) on its charging cable, showing a pen's newest specimen with its photograph and numbered evidence markers (b), the evidence in words (c), and the pen's five appearance checks against the barn average (d). The display is shown with the dashboard's demonstration data, not a field reading.

Table 1: Accuracy and macro-F1 on the held-out farm test (679 photographs) and the lab-confirmed photographs (1,255), for models trained on the same 5,451 photographs. One training run per model. The detector is scored with its best class in each photograph (always answers) and with the deployed 0.25 confidence threshold (below it the photograph counts as wrong).

	Farm test		Lab-confirmed	
Model	Accuracy	Macro-F1	Accuracy	Macro-F1
MobileNetV2 [25]	99.6	99.7	74.6	75.3
EfficientNet-B0 [11]	99.0	98.6	78.2	79.5
YOLO11n (ours), always answers	98.5	98.3	76.3	75.1
YOLO11n (ours), 0.25 threshold	97.9	97.7	67.1	70.3

drop. The models fail on different classes. The classifiers keep far more Newcastle-like photographs (recall 0.70 for MobileNetV2 and 0.88 for EfficientNet-B0, against 0.53 for the detector), while the detector keeps more healthy ones (0.81 against 0.66 and 0.68). The per-class differences are large, but the differences in overall accuracy are small next to the drop that every model shows. Under this protocol the published accuracies of 97.8 to 99.3% describe photographs like the training photographs, and the architecture matters less than the test set.

4.2 Where the errors go

Figure 4 shows the detector's calls on both sets. On the farm test the diagonal holds 95 to 100% of every class. On the lab-confirmed photographs most errors are not confusions between diseases. They are photographs with no detection at all (20% of healthy and 27% of Newcastle-like photographs) and photographs called healthy or salmonella-like by mistake. Newcastle-like recall falls the most, from 94.6% (35 of 37) to 41.9% (78 of 186). It has 376 farm photographs, against over 2,000 for each other class (Figure 5).

4.3 Duplicates do not explain the farm score

Forty-nine of the 679 farm test photographs (7.2%) have a training photograph with an identical 256-bit difference hash (Figure 6). Each pair has the same dimensions and class, and the pair we inspected by eye is the same image saved under two file names. Every other test photograph differs from its nearest training photograph in at least 40 bits. The validation split is affected too: 53 validation photographs copy a training photograph, and validation chose the kept epoch. The detector calls all 49 test copies correctly. Without them its farm accuracy is 97.8% (95% CI 96.3 to 98.7%) at the 0.25 threshold, against 97.9% with them, and the other rows of Table 1 change by at most 0.1 points (detector always answering 98.4%, MobileNetV2 99.5%, EfficientNet-B0 99.1%). The copies are a flaw in the split, but they are not why the farm test overstates performance on new photographs.

4.4 The box score ranks well but is not a probability

If the detector only answers when its best box clears a threshold, its answers become more reliable (Figure 7). On the lab-confirmed photographs, raising the threshold from 0.25 to 0.80 raises the accuracy of the answers from 78.8% to 87.4%, but the detector then answers only 27.8% of photographs. Its scores are also under-confident on both sets (Figure 8). The expected calibration error is 0.177 on the farm test and 0.172 on the lab-confirmed photographs, and in both cases the photograph is right more often than the score says. The score should be cal-

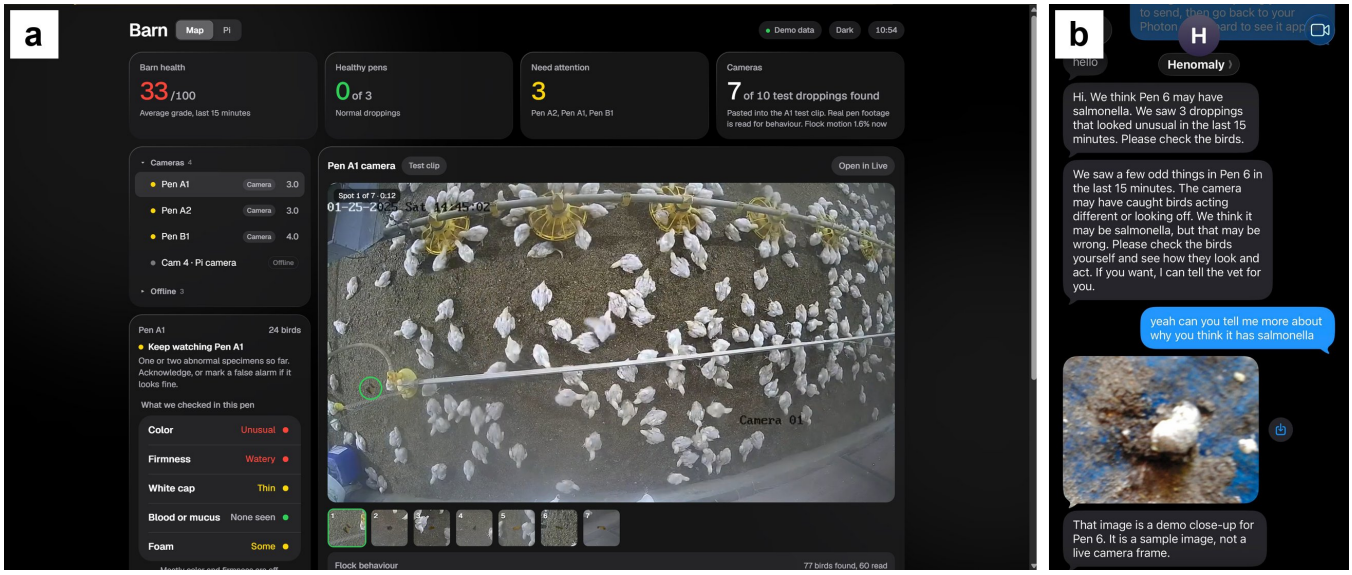


Figure 2: The two other interfaces. (a) The dashboard with the held-out Pen A1 test clip, in which test droppings were pasted at known times. A green ring marks a flagged spot and the strip below lists the seven spots found. Pen readings on the dashboard are demonstration data, as its header states. (b) The farmer messaging service in a test conversation. The attached photograph is a sample image, as the agent itself says, not a live camera frame.

Table 2: Appearance grade and model accuracy (top class equals the PCR class) on the lab-confirmed photographs. Normal looking means grade 9 or 10. n excludes the nine photographs no rater could judge. Detector is our YOLO11n scored on its top class (always answers).

PCR class	n	Mean grade	Normal looking	Acc. on normal looking Detector	MobileNetV2
Healthy	347	8.6	62%	84%	66%
Salmonella	347	8.0	47%	93%	98%
Coccidiosis	366	5.6	7%	23%	23%
Newcastle	186	4.6	1%	1 photograph	

ibrated, for example by temperature scaling [15] on the validation split, before a farmer sees it as a percentage.

4.5 What the lab-confirmed droppings look like

The three raters agreed well on the grade, with Krippendorff's alpha 0.76 over the 1,255 photographs and pairwise weighted kappa from 0.71 to 0.84. Agreement on single variables was lower, from 0.56 for colour and 0.57 for undigested feed to 0.75 for the urate cap and 0.74 for blood. Each rater agreed with itself on the 126 repeats with weighted kappa 0.71 to 0.86, but gave the identical grade only 37 to 47% of the time, so a single grade is precise to about one point. Nine photographs showed no dropping that could be judged, 331 were only partly visible, and on 234 the raters' grades spread by three points or more. The grades, every rating with its rater's note and these agreement figures are released with the paper.

The grade follows the PCR class (Table 2). Newcastle disease and coccidiosis droppings look abnormal, with mean grades 4.6 and 5.6, while salmonella droppings are close to healthy ones, 8.0 against 8.6, and nearly half of them look normal. The raters scored salmonella droppings as looser (mean form penalty 0.61 against 0.28) and with more undigested feed (0.44 against 0.09).

We expected the models to fail most on infected birds whose droppings look normal. They do not. Over the 899 PCR-positive photographs, the detector is right on 83% of the 190 that look normal and on 73% of the 709 that do not, and MobileNetV2 on 87% and 76%. The exception is coccidiosis, where both models are right on only 6 of the 26 normal looking photographs. The high score on normal looking positives comes from salmonella, which both models call correctly 93 to 98% of the time even when the raters saw nothing abnormal. Part of that is a lean towards salmonella, since MobileNetV2 also calls 72 of the 215 normal looking healthy photographs salmonella (33%), and the detector 32 (15%). Telling a normal looking salmonella dropping from a healthy one by eye is hard, so cues other than the dropping's clinical signs, such as the surface, lighting or the camera used for one class, may be helping. These grades cannot show which.

4.6 Pen-scale detection

The close-up detector does not work at pen scale (Figure 9). On the held-out pen cameras it finds 29% of droppings, only 11% of its boxes are droppings, and it places 3.55 boxes on an average empty pen tile at the 0.25 threshold. After pen-scale fine-tuning, recall is 0.80 and precision 0.93 on cameras the model never saw, and

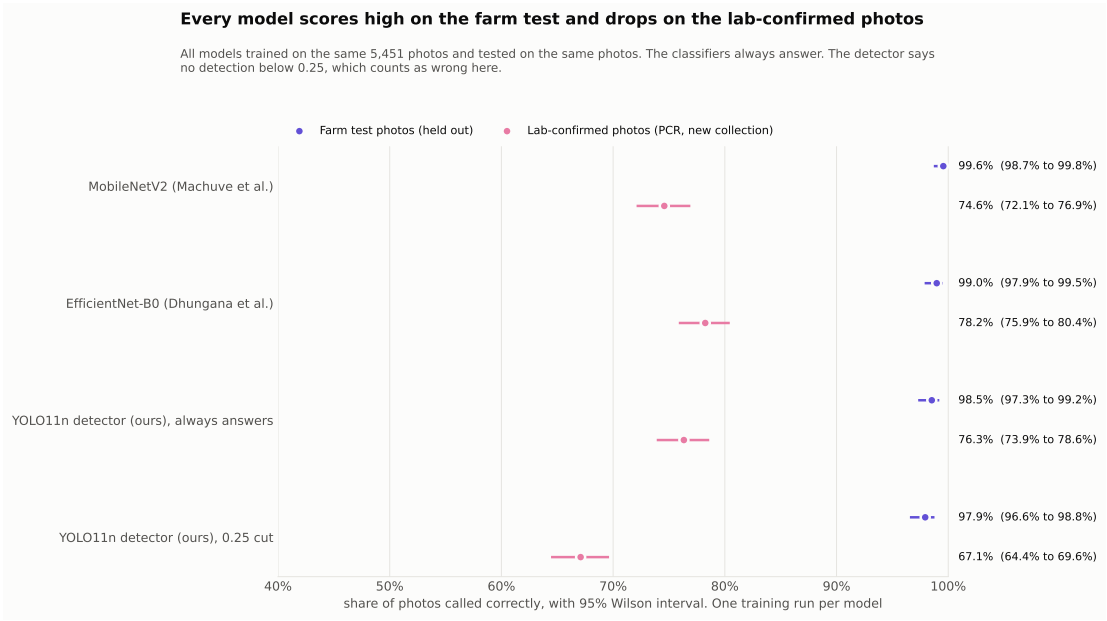


Figure 3: Accuracy of each model on the farm test and on the lab-confirmed photographs. One training run per model. Values are written beside each mark.

false boxes on empty tiles fall to 0.05 per tile. The close-up mAP50 changes from 0.873 to 0.853. Class accuracy at pen size remains weaker than detection (four-class mAP50 0.65). The pen model should therefore find a dropping, and a full-resolution patch or a close-up photograph should be judged by the close-up model.

Whole-photograph classifiers cannot be tested this way. They return one label for an image, and in a pen frame a dropping covers at most about 0.1% of the image. Locating the dropping is what makes the pen camera usable, and it is the main reason to use a detector in this setting rather than a better classifier.

4.7 The deployed pipeline

We also ran the pen model over a three-minute clip from the held-out PIO camera, into which ten test droppings were pasted at known times. Read at two frames a second with a short persistence rule, it flagged seven, five of them within two seconds of their appearance, and raised no false flags (Figure 10). This run was made on a laptop, and like the pen test it uses pasted droppings. On the Pi, the deployed pipeline proposes candidates with a still-spot finder and judges a full-resolution patch of each with the close-up detector. On the Raspberry Pi 5 the detector takes 168 ms at 320 pixels and 618 ms at 640 pixels with four threads, and the pen model 608 ms at 640 pixels. In the deployed pipeline the detector runs only on the candidate patches the spot finder flags, not on every frame. We did not time the baseline classifiers on the Pi, and we have not measured the detection accuracy of the deployed pipeline on the Pi. Native message passing between two sections takes 95 μs for a 57.6 KB frame and reply. When we killed a section with `SIGKILL`, the supervisor started a new copy of it within 20 to 33 ms. This is the time from the supervisor noticing the exit to the new process being launched. The supervisor checks its children every 100 ms, so a stage can be down for up to about 130 ms in total, plus the time the new process needs to start serving. We ran these kills by hand on each section during development and did not keep a count per section. Killing the spot finder cost the camera one frame and the other sections none. These kills test our process structure and supervisor, which would work the same way on Linux. We did not kill a QNX driver or system service, such as the camera service or the network stack.

5 Discussion

What the farm test measures. Every model we trained, including the two that earlier work recommended, reaches 98.5 to 99.6% on the random farm test and loses 21 to 25 points on photographs from a second collection of the same group. The original authors did not expect distribution shift to matter [25]. Our results show that it matters more than the choice of network. We cannot say from these data which difference between the collections causes the drop, but the farm collection itself suggests a strong candidate. Its salmonella and Newcastle photographs come from experimentally inoculated birds and its healthy and coccidiosis photographs from farms [24], while every PCR-labelled photograph comes from a farm. A model can then learn the setting of each class, such as the surface, the lighting or the birds, along with the dropping, which is the kind of shortcut described by Geirhos et al. [13]. Healthy drop-

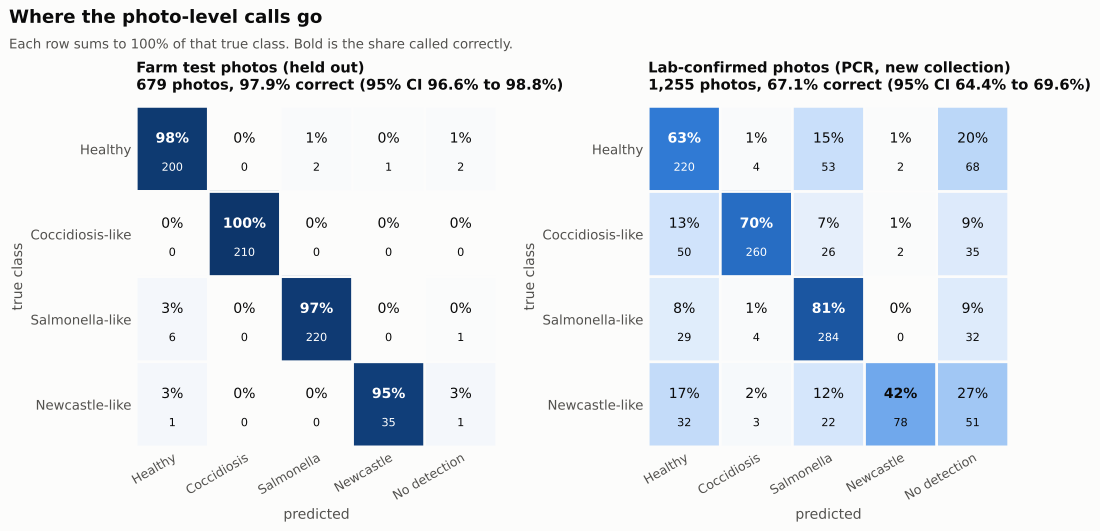


Figure 4: Detector calls per true class on the farm test (left) and the lab-confirmed photographs (right). Each row sums to 100%. Counts under each share.

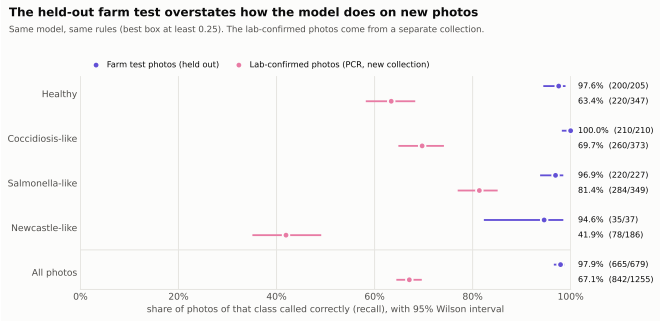


Figure 5: Per-class recall of the detector with 95% Wilson intervals, at the deployed 0.25 threshold.

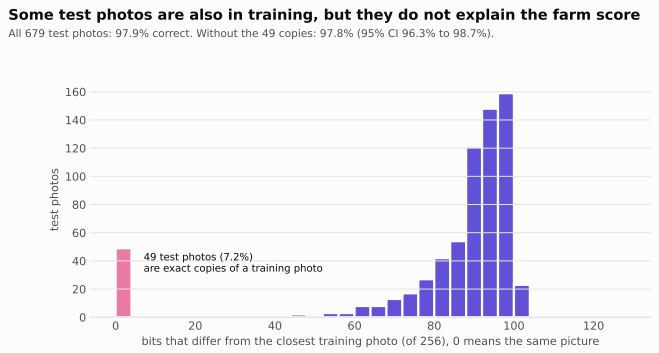


Figure 6: Hamming distance from each farm test photograph to its nearest training photograph (256-bit difference hash). The bar at zero holds the 49 exact copies.

pings in the farm collection also fill a median of 2.6% of the photograph, against 12 to 22% for the other classes, so framing could act as a shortcut too. Three observations fit this reading without proving it. Exact duplicates are not the cause, because removing them leaves the farm score unchanged, and neither are the conflicting PCR files, which move every model by about one point. Most errors are missed detections and calls of healthy or salmonella-like, not swaps between the diseases. And on normal-looking PCR-labelled photographs the models lean towards salmonella (Section 4.5), the class whose training photographs came from one inoculation setting. We have not tested these shortcuts directly. Sakib et al. saw a much smaller drop, from 99.19 to 95.62%, on a local-farm set [30]. Their abstract does not say how that set was labelled or collected, so we cannot tell whether it shares the setting of the training photographs. Bodempudi et al. reached at most 85% across regions for a binary *Salmonella* task [6]. Together these results suggest that published accuracies on this task should be read as within-collection accuracies.

Why use a detector. On photographs our detector is not more accurate than a whole-image classifier. Its value is that it says where the dropping is. That is what makes a fixed pen camera usable, because under our size assumption a dropping covers at most about 0.1% of a pen frame, and it is also what lets the explanation step measure the colour of the dropping itself rather than the litter around it. The pen-scale fine-tune shows that the detector can learn the size and setting of a pen camera from public video and pasted droppings alone, without a single labelled pen dropping. It lost 0.02 mAP50 on close-ups in the process. The Pi pipeline does not use the pen model yet. It finds candidates with a still-spot finder and judges them with the close-up model, whose false alarms on white birds the explanation step filters out.

Why a microkernel. A monitor in a barn runs unattended, and a crash in a camera driver or a model runtime should cost a few frames, not the device. On QNX each

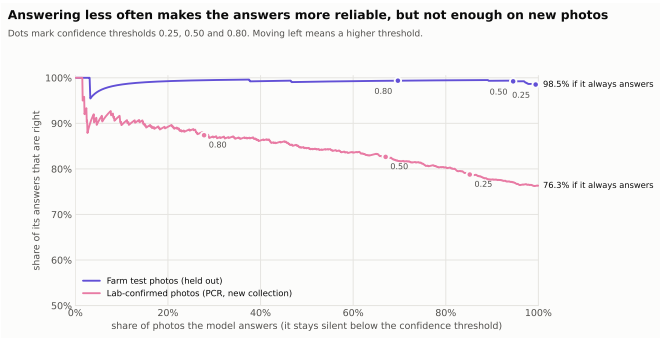


Figure 7: Accuracy of the detector's answers against the share of photographs it answers, as the confidence threshold rises. Dots mark the thresholds 0.25, 0.50 and 0.80.

stage of our pipeline is an ordinary process, and the supervisor relaunched any stage we killed within 20 to 33 ms of noticing the exit while the others kept running. The same structure is possible on Linux with process supervision. What QNX adds is that the drivers, the file system and the network stack are also user processes that can fail and restart the same way [3], but our tests killed only our own stages, not a QNX driver or service. We did not compare fault recovery against Linux on the same board, so we do not claim QNX recovers faster. Running the model on QNX also needed one fix in the platform's packages. The Python binding of ONNX Runtime in the QNX open-source repository does not load, because its build script treats QNX as Windows and looks for the wrong extension module. We worked around it with a 111-line C bridge to the runtime's C library and have prepared a patch for the package recipe.

Limitations. All photographs come from one region of Tanzania and one research group. No farm photograph carries a laboratory diagnosis of its own dropping, two farm classes come from inoculated birds, and the PCR release contains files filed under two classes, so we treat the classes as screening categories, not diagnoses. The farm split is random by photograph because the release carries no farm or visit information. The pen test and the test clip use real droppings pasted onto real barn frames, so they measure whether a pen-sized dropping can be found on unseen litter, not field accuracy on naturally occurring droppings. The pen size of 8 to 30 pixels is an assumption. We have no labelled droppings from a real pen camera, and we have not measured the detection accuracy of the deployed pipeline. The detector and each baseline were trained once, with one seed and one set of hyperparameters that we did not tune, and EfficientNet-B0 for only 8 epochs. Run-to-run variation is therefore not measured. To explain the drop it would have to exceed 20 points, far beyond the 0.02 mAP50 gap between our two detector runs on the farm test. Finally, the edge timings are for inference only and do not include image decoding, post-processing or power draw. The appearance grades come from three AI models following a rubric, not from veterinarians. Their agreement shows the grades are consistent, not that they are right, and three models from one vendor may share blind spots. The rubric is new and has not been validated against expert grading.

Future work. The most useful next step is labelled droppings from fixed pen cameras on more than one farm, split by farm. Short of that, the farm collection could be re-split with near-duplicate removal before splitting, the size shortcut could be tested by cropping every class to the same framing, and the setting shortcut by testing on salmonella and Newcastle photographs taken on farms. On the device, the next steps are a Linux baseline on the same Raspberry Pi 5 and fault tests that kill QNX services, not only our own stages. The box score needs calibration before it is shown to a farmer, and the 0.25 threshold should be chosen from a risk-coverage curve on validation data rather than fixed.

6 Conclusion

Classifiers of poultry droppings that score 98.5 to 99.6% on a random split of the Tanzanian farm photographs score 75 to 78% on PCR-labelled photographs from a second collection taken on farms, and this holds for every architecture we tried, including our detector. We therefore recommend that future work on this task report a second collection alongside the farm test. A detector cannot fix this drop, but it is what makes a fixed pen camera usable. Fine-tuned on public barn video with pasted droppings, our YOLO11n detector finds 80% of pen-sized droppings on unseen cameras with 93% precision. A pipeline built around the close-up detector runs on a Raspberry Pi 5 under QNX, where a stage killed with `SIGKILL` is relaunched within tens of milliseconds while the rest keeps running.

Data and code availability

The farm and lab-confirmed photographs are Zenodo records 4628934 and 5801834, the PIO and NESTLER videos Zenodo records 16686320 and 15063494, all under CC BY 4.0. Our code, trained weights, per-photograph predictions, duplicate hashes, appearance grades with every rating, training logs and every figure with the table behind it are at `https://github.com/RishitRai/Holy-Crap`. The figures are produced by `data-visualization/evaluation.py`, the baselines by `training/benchmark_classifiers.py`, and the image audit by `data-visualization/audit_images.py`.

Ethics

This study used only published photographs and videos. No animals were handled and no new data were collected.

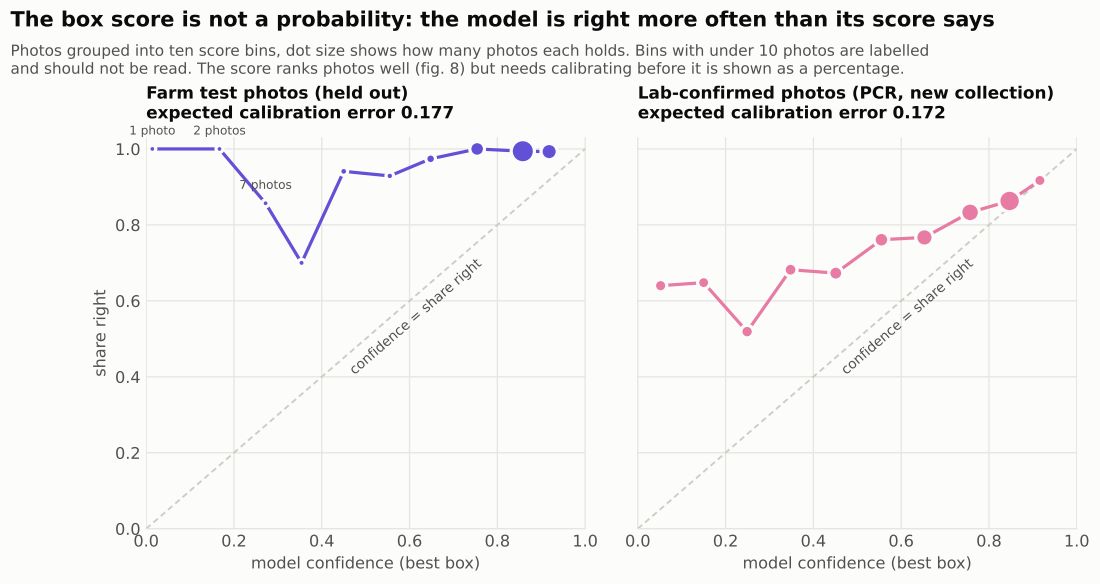


Figure 8: Reliability diagrams over ten score bins. Dot size shows the number of photographs in a bin. Bins with fewer than ten photographs are labelled and should not be read.

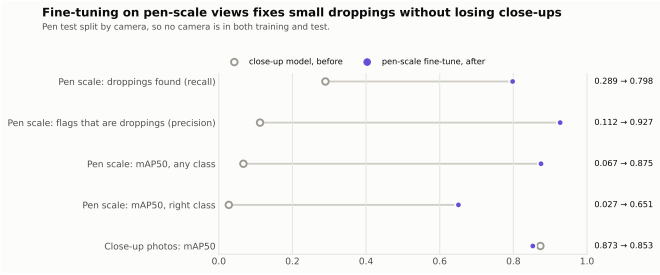


Figure 9: Pen-scale test on held-out cameras before (close-up model) and after fine-tuning. The pen test uses real droppings pasted onto real litter.

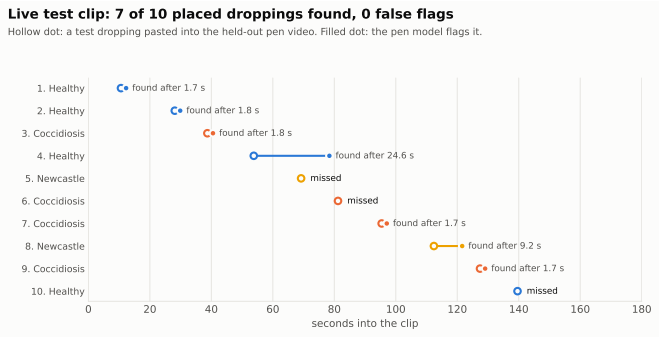


Figure 10: Test clip with an answer key. Hollow dots mark when each pasted dropping appears in the held-out video, filled dots when the pen model flagged it.

Compute

The detector, the pen-scale fine-tune and the baselines were trained on a local laptop GPU. The pen-scale fine-tune, the two baselines and every evaluation in this paper ran on an NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB of memory. The replicate detector run was trained on the Veri cloud platform on one NVIDIA A10G GPU (job `abe1b30a56c4`). All on-device timings were measured on a Raspberry Pi 5 running QNX 8.0.

Use of AI tools

The appearance grades in Section 3.7 were produced by three AI models, as described there. AI coding and writing assistants were used to write analysis code and to draft and edit the text. The authors checked every number against the released result files and every reference against its source, and take responsibility for the content.

Acknowledgements

We thank Machuve and colleagues for releasing the farm and PCR-confirmed photographs, and the authors of the PIO and NESTLER datasets for releasing their barn videos, all under open licences. We thank the QNX team for the QNX Everywhere program and the open-source package repository at `https://oss.qnx.com`, whose ONNX Runtime, Python and numpy builds made on-device inference possible, and Veri for the cloud GPU time used for the replicate training run. This work was carried out at LA Hacks, in its QNX and Veri tracks. We also thank the developers of Ultralytics YOLO, PyTorch and ONNX Runtime.

References

[1] Halleluyah O. Aworinde, Segun Adebayo, Akinwale O. Akinwunmi, Olufemi M. Alabi, Adebamiji Ayandiji, Aderonke B. Sakpere, Abel K. Oyebamiji, Oke Olaide, Ezenma Kizito, and Abayomi J. Olawuyi. Poultry fecal imagery dataset for health status prediction: A case of south-west nigeria. *Data in Brief*, 50:109517, 2023. doi: 10.1016/j.dib.2023.109517.

[2] Björn Barz and Joachim Denzler. Do we train on test data? purging CIFAR of near-duplicates. *Journal of Imaging*, 6(6):41, 2020. doi: 10.3390/jimaging6060041.

[3] BlackBerry Limited. *QNX Software Development Platform 8.0: System Architecture*. BlackBerry QNX. URL `https://www.qnx.com/developers/docs/8.0/com.qnx.doc.neutrino.sys_arch/topic/about.html`. Accessed 2026-10-18.

[4] BlackBerry QNX. QNX for deterministic physical AI workloads: A comparative benchmark of embedded NPU inference performance on Hailo-8. White Paper 02, BlackBerry QNX, 2026. URL `https://qnx.software/content/dam/qnx-xwalk/pdf/white-papers/embedded-systems/qnx-hailo-physical-ai-whitepaper.pdf`. Vendor-authored benchmark.

[5] Damer P. Blake, Jolene Knox, Ben Dehaeck, Ben Huntington, Thilak Rathinam, Venu Ravipati, Simeon Ayoade, Will Gilbert, Ayotunde O. Adebambo, Isa Danladi Jatau, Muthusamy Raman, Daniel Parker, Jonathan Rushton, and Fiona M. Tomley. Re-calculating the cost of coccidiosis in chickens. *Veterinary Research*, 51(1):115, 2020. doi: 10.1186/s13567-020-00837-2.

[6] Venkat Umesh Chandra Bodempudi, Guoming Li, Adelumola Oladeinde, Michael J. Rothrock, Samuel E. Aggrey, Tongshuai Liu, Sai Akshitha Reddy Kota, Geng Yuan, and Sravan Sai Rahul Nalla. Salmonella risk prediction in poultry farms via deep learning image classification models, cross-region validation, and edge computing. *Computers and Electronics in Agriculture*, 252:112116, 2026. doi: 10.1016/j.compag.2026.112116.

[7] Keyla Boniche and Edmanuel Cruz. PIO, a large-scale dataset for broiler chicken detection under real poultry farming conditions DATASET v.1, 2025. Dataset. License: CC BY 4.0.

[8] Keyla Boniche, Edmanuel Cruz, José Carlos Rangel, Miguel Hidalgo-Rodríguez, and Francisco Gómez-Donoso. PIO, a large-scale dataset for broiler chicken detection under real poultry farming conditions. *Scientific Data*, 13(1):801, 2026. doi: 10.1038/s41597-026-07114-5.

[9] Jacob Cohen. Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit. *Psychological Bulletin*, 70(4):213–220, 1968. doi: 10.1037/h0026256.

[10] Mizanu Zelalem Degu and Gizeaddis Lamesgin Simegn. Smartphone based detection and classification of poultry diseases from chicken fecal images using deep learning techniques. *Smart Agricultural Technology*, 4:100221, 2023. doi: 10.1016/j.atech.2023.100221.

[11] Anjan Dhungana, Xiao Yang, Bidur Paneru, Samin Dahal, Guoyu Lu, and Lilong Chai. An integrated deep learning approach for poultry disease detection and classification based on analysis of chicken manure images. *AgriEngineering*, 7(9):278, 2025. doi: 10.3390/agriengineering7090278.

[12] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL `https://papers.nips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html`.

[13] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. doi: 10.1038/s42256-020-00257-z.

[14] Golnaz Ghiasi, Yin Cui, Aravind Srinivas, Rui Qian, Tsung-Yi Lin, Ekin D. Cubuk, Quoc V. Le, and Barret Zoph. Simple copy-paste is a strong data augmentation method for instance segmentation. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2917–2927, 2021. doi: 10.1109/CVPR46437.2021.00294.

[15] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330, 2017. URL `https://proceedings.mlr.press/v70/guo17a.html`.

[16] Andrew F. Hayes and Klaus Krippendorff. Answering the call for a standard reliability measure for coding data. *Communication Methods and Measures*, 1(1):77–89, 2007. doi: 10.1080/19312450709336664.

[17] Md Sadi Al Huda, Adety Sarkar, Kazi Tanvir, Md Asraf Ali, Dip Nandi, and Mohamed Haq. Enhancing poultry disease classification using fecal image: A fusion approach. In *Proceedings of the 3rd International Conference on Computing Advancements*, pages 830–837. ACM, 2024. doi: 10.1145/3723178.3723288.

[18] Glenn Jocher and Jing Qiu. Ultralytics YOLO11, 2024. URL `https://github.com/ultralytics/ultralytics`.

[19] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023. doi: 10.1016/j.patter.2023.100804.

[20] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*, pages 207–220. ACM, 2009. doi: 10.1145/1629575.1629596.

[21] Neal Krawetz. Kind of like that. The Hacker Factor Blog, 2013. Describes the difference hash (dHash).

[22] Guoming Li. A survey of open-access datasets for computer vision in precision poultry farming. *Poultry Science*, 104(2):104784, 2025. doi: 10.1016/j.psj.2025.104784.

[23] Dina Machuve, Ezinne Nwankwo, Emmanuel Lyimo, Evarist Maguo, and Charles Munisi. Machine learning dataset for poultry diseases diagnostics - PCR annotated, 2021. Dataset. License: CC BY 4.0.

[24] Dina Machuve, Ezinne Nwankwo, Neema Mduma, Hope Mbelwa, Evarist Maguo, and Charles Munisi. Machine learning dataset for poultry diseases diagnostics, 2021. Dataset. License: CC BY 4.0.

[25] Dina Machuve, Ezinne Nwankwo, Neema Mduma, and Jimmy Mbelwa. Poultry diseases diagnostics models using deep learning. *Frontiers in Artificial Intelligence*, 5:733345, 2022. doi: 10.3389/frai.2022.733345. Brief Research Report.

[26] Arnas Nakrosis, Agne Paulauskaite-Taraseviciene, Vidas Raudonis, Ignas Narusis, Valentas Gruzauskas, Romas Gruzauskas, and Ingrida Lagzdinyte-Budnike. Towards early poultry health prediction through non-invasive and computer vision-based dropping classification. *Animals*, 13 (19):3041, 2023. doi: 10.3390/ani13193041.

[27] NESTLER Horizon Project. NESTLER poultry video dataset, 2025. Video dataset. License: CC BY 4.0. EU Horizon Grant Agreement no. 101060762.

[28] ONNX Runtime developers. ONNX Runtime, 2018. URL `https://onnxruntime.ai`. Version 1.24.4 used. Code: https://github.com/microsoft/onnxruntime. License: MIT.

[29] Al-Momen Reyad, Sonjoy Dey, Wasi Uddin Bhuyian, Sahadat Islam Evan, Nusrat Jahan, and Raiyan Rahman. An ensemble deep learning approach to detect common chicken diseases from fecal matter images. In *Proceedings of the 3rd International Conference on Computing Advancements*, pages 145–153. ACM, 2024. doi: 10.1145/3723178.3723198.

[30] Shahbaj Ahmed Sakib, Md Mursalin Hossain Misat, Tanjum Rahi Akanto, Jahedul Islam, and Fahmida Ahmed Antara. IoT enabled smart poultry farming system with deep learning for chicken health detection in real-time. *Journal of Sensors*, 2026(1): 1433795, 2026. doi: 10.1155/js/1433795.

[31] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018.

[32] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pages 6105–6114, 2019.

[33] Jintao Wang, Mingxia Shen, Longshen Liu, Yi Xu, and Cedric Okinda. Recognition and classification of broiler droppings based on deep convolutional neural network. *Journal of Sensors*, 2019:1–10, 2019. doi: 10.1155/2019/3823515. Article ID 3823515.

[34] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.

[35] Qianxue Zhang and Eiman Kanjo. A multicore and edge TPU-accelerated multimodal TinyML system for livestock behavior recognition. *IEEE Internet of Things Journal*, 13(1):666–677, 2026. doi: 10.1109/JIOT.2025.3624811.